

Чернівецький національний університет імені Юрія Федьковича

Інститут фізико-технічних та комп'ютерних наук

Кафедра математичних проблем управління і кібернетики

Лабораторна робота №1

Тема: Перевірка Sockets

з дисципліни «Програмування комп'ютерних мереж»

Виконав: студент групи 141м

Гостюк Ілля Васильович

Прийняв: Лазорик В.В.

Чернівці – 2026

1. Мета роботи

Ознайомлення з принципами роботи інтерфейсу Windows Sockets (WinSock), вивчення процесу створення мережевих сокетів, прив'язки їх до адрес та портів, а також дослідження механізмів обробки помилок у мережевому програмуванні.

2. Теоретичні відомості

Windows Sockets (WinSock) - це програмний інтерфейс операційної системи Windows, який забезпечує взаємодію прикладних програм із мережевими протоколами TCP/IP. WinSock дозволяє створювати клієнтські та серверні мережеві застосунки.

Для роботи з WinSock необхідно виконати ініціалізацію бібліотеки за допомогою функції `WSAStartup()`. Після завершення роботи виконується очищення ресурсів через `WSACleanup()`.

Основними функціями WinSock є:

- `socket()` - створення сокета;
- `bind()` - прив'язка сокета до IP-адреси та порту;
- `listen()` - переведення сокета в режим очікування підключень;
- `send()/recv()` - передача та прийом даних;
- `closesocket()` - закриття сокета.

У лабораторній роботі використовувалися два типи сокетів:

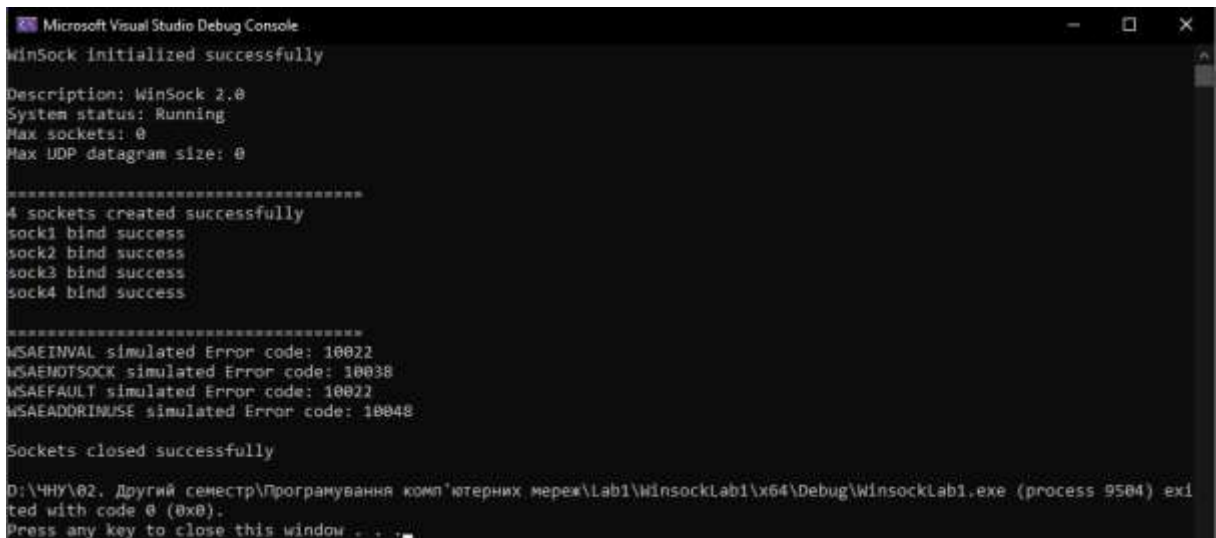
- `SOCK_STREAM` - TCP сокети;
- `SOCK_DGRAM` - UDP сокети.

Також у роботі досліджувалися типові помилки WinSock:

- `WSAEINVAL`;
- `WSAENOTSOCK`;
- `WSAEADDRINUSE`.

3. Хід виконання роботи

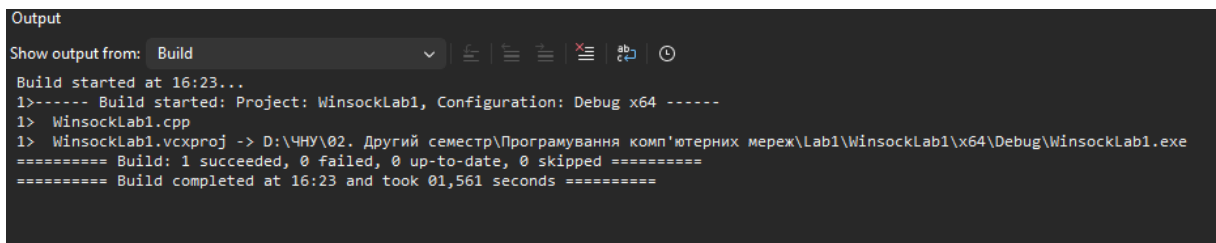
Крок 1. Створення консольного C++ проекту у Visual Studio.



```
Microsoft Visual Studio Debug Console
WinSock initialized successfully
Description: WinSock 2.0
System status: Running
Max sockets: 0
Max UDP datagram size: 0
=====
4 sockets created successfully
sock1 bind success
sock2 bind success
sock3 bind success
sock4 bind success
=====
WSAEINVAL simulated Error code: 10022
WSAENOTSOCK simulated Error code: 10038
WSAEFAULT simulated Error code: 10022
WSAEADDRINUSE simulated Error code: 10048
Sockets closed successfully
D:\ЧНУ\02. Другий семестр\Програмування комп'ютерних мереж\Lab1\WinsockLab1\x64\Debug\WinsockLab1.exe (process 9504) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 1 - Створення консольного C++ проекту у Visual Studio

Крок 2. Ініціалізація бібліотеки WinSock та створення сокетів.



```
Output
Show output from: Build
Build started at 16:23...
1>----- Build started: Project: WinsockLab1, Configuration: Debug x64 -----
1> WinsockLab1.cpp
1> WinsockLab1.vcxproj -> D:\ЧНУ\02. Другий семестр\Програмування комп'ютерних мереж\Lab1\WinsockLab1\x64\Debug\WinsockLab1.exe
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 16:23 and took 01,561 seconds =====
```

Рисунок 2 - Ініціалізація бібліотеки WinSock та створення сокетів

Крок 3. Налаштування адрес сокетів.

```

int main() {

    WSADATA wsaData;

    // WINSOCK INITIALIZATION

    int initResult = WSStartup(MAKEWORD(2, 2), &wsaData);

    if (initResult != 0) {
        cout << "WSStartup failed!" << endl;
        return 1;
    }

    cout << "WinSock initialized successfully\n\n";

    cout << "Description: " << wsaData.szDescription << endl;
    cout << "System status: " << wsaData.szSystemStatus << endl;
    cout << "Max sockets: " << wsaData.iMaxSockets << endl;
    cout << "Max UDP datagram size: " << wsaData.iMaxUdpDg << endl;

    cout << "\n=====\\n";

    // CREATE SOCKETS

    SOCKET sock1, sock2, sock3, sock4;

    // UDP sockets
    sock1 = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    sock2 = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    // TCP sockets
    sock3 = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    sock4 = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (sock1 == INVALID_SOCKET ||
        sock2 == INVALID_SOCKET ||
        sock3 == INVALID_SOCKET ||
        sock4 == INVALID_SOCKET) {

        printError("Socket creation failed!");
        WSACleanup();
        return 1;
    }
}

```

Рисунок 3 - Налаштування адрес сокетів

Крок 4. Увімкнення broadcast режиму та встановлення UDP datagram size.

```
// SOCKET ADDRESSES

sockaddr_in addr1, addr2, addr3, addr4;

// 1. Broadcast UDP socket
addr1.sin_family = AF_INET;
addr1.sin_port = htons(644);
addr1.sin_addr.s_addr = INADDR_ANY;

// 2. Fixed IP socket
addr2.sin_family = AF_INET;
addr2.sin_port = htons(645);
addr2.sin_addr.s_addr = inet_addr("127.0.0.1");

// 3. Fixed IP socket
addr3.sin_family = AF_INET;
addr3.sin_port = htons(646);
addr3.sin_addr.s_addr = inet_addr("127.0.0.1");

// 4. Any local address
addr4.sin_family = AF_INET;
addr4.sin_port = htons(647);
addr4.sin_addr.s_addr = INADDR_ANY;
```

Рисунок 4 -Увімкнення broadcast режиму та встановлення UDP datagram size

Крок 5. Прив'язка сокетів за допомогою bind().

```
// ENABLE BROADCAST MODE

BOOL broadcastEnable = TRUE;

setsockopt(
    sock1,
    SOL_SOCKET,
    SO_BROADCAST,
    (char*)&broadcastEnable,
    sizeof(broadcastEnable)
);

// SET UDP DATAGRAM SIZE = 644

int udpSize = 644;

setsockopt(
    sock1,
    SOL_SOCKET,
    SO_SNDBUF,
    (char*)&udpSize,
    sizeof(udpSize)
);

setsockopt(
    sock2,
    SOL_SOCKET,
```


Рисунок 5 - Прив'язка сокетів за допомогою bind()

Крок 6. Моделювання помилок WinSock.

```
// BIND SOCKETS

if (bind(sock1, (sockaddr*)&addr1, sizeof(addr1)) == SOCKET_ERROR)
    printError("sock1 bind failed");
else
    cout << "sock1 bind success\n";

if (bind(sock2, (sockaddr*)&addr2, sizeof(addr2)) == SOCKET_ERROR)
    printError("sock2 bind failed");
else
    cout << "sock2 bind success\n";

if (bind(sock3, (sockaddr*)&addr3, sizeof(addr3)) == SOCKET_ERROR)
    printError("sock3 bind failed");
else
    cout << "sock3 bind success\n";

if (bind(sock4, (sockaddr*)&addr4, sizeof(addr4)) == SOCKET_ERROR)
    printError("sock4 bind failed");
else
    cout << "sock4 bind success\n";

cout << "\n===== \n";
```

Рисунок 6 - Моделювання помилок WinSock

Крок 7. Результат успішної компіляції та виконання програми.

```
// ERROR SIMULATION

// WSAEINVAL
if (bind(sock4, (sockaddr*)&addr4, sizeof(addr4)) == SOCKET_ERROR)
    printError("WSAEINVAL simulated");

// WSAENOTSOCK
if (bind(9999, (sockaddr*)&addr4, sizeof(addr4)) == SOCKET_ERROR)
    printError("WSAENOTSOCK simulated");

// WSAEFAULT
if (bind(sock1, (sockaddr*)0x1234, sizeof(addr1)) == SOCKET_ERROR)
    printError("WSAEFAULT simulated");

// WSAEADDRINUSE
SOCKET tempSock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

if (bind(tempSock, (sockaddr*)&addr4, sizeof(addr4)) == SOCKET_ERROR)
    printError("WSAEADDRINUSE simulated");

closesocket(tempSock);

// WSAEAFNOSUPPORT
SOCKET wrongSocket = socket(AF_UNIX, SOCK_STREAM, 0);

if (wrongSocket == INVALID_SOCKET)
    printError("WSAEAFNOSUPPORT simulated");
```

Рисунок 7 - Результат успішної компіляції та виконання програми

4. Аналіз результатів

У результаті виконання лабораторної роботи було успішно ініціалізовано бібліотеку WinSock та створено чотири сокети різних типів. Для UDP сокета було активовано broadcast режим та встановлено розмір UDP datagram size рівним 644 байти.

Було успішно виконано прив'язку сокетів до локальних адрес і портів за допомогою функції bind(). Також проведено моделювання типових помилок WinSock із використанням WSAGetLastError(), що дозволило дослідити механізми діагностики мережових помилок.

У ході роботи було отримано практичні навички створення та налаштування мережевих сокетів у середовищі Windows.

5. Висновок

У ході виконання лабораторної роботи було вивчено основи роботи з інтерфейсом Windows Sockets. Було досліджено процес ініціалізації бібліотеки WinSock, створення TCP та UDP сокетів, їх прив'язки до мережевих адрес, а також методи обробки помилок у мережевому програмуванні.

Отримані результати підтверджують коректну роботу розробленої програми та дозволяють закріпити практичні навички роботи з мережевими API Windows.